

App

Must Have

- Input natal chart data (date, time, location) and generate astrological chart.
- Integration of ephemeris to analyze future planetary movements.
- Personal spirit/Orisha database with user-defined correspondences.
- User selects magical goal (e.g., protection, abundance) and receives:
 - Recommended spirits aligned with goal.
 - Ideal election time with astrological justification.
 - Symbolic prescription: shapes, colors, glyphs, numbers, materials.
- Visual builder to preview talisman layout with geometry, color, symbols.
- Exportable "Talisman Blueprint" (structured data + visual).
- Store all data locally/private for single-user use.
- Transit Watcher: Detect upcoming major transits that impact user's natal chart (e.g., hard Saturn square Moon), and:
 - Classify as Positive / Challenging.
 - Recommend talismans that either harness or shield from those influences.

Should Have

- Learning section on relevant astrological & symbolic systems.
- Review past talismans and linked election data.
- Filter/search spirit database by goal, planet, element, etc.
- Planetary hour and lunar mansion tracking.

Could Have

- Export to Illustrator/Inkscape-ready SVG.
- Spirit tagging/grouping for easier access (e.g., ancestral, solar, malefic).
- Election calendar sorted by goal and planetary rulership.

Won't Have (in MVP)

- User authentication or shared access.
- Public spirit exchange or community forums.
- AR previewing or mobile app.
- AI-based synthesis of new correspondences.

Method

System Architecture Overview

```
@startuml
package "Astrological Talisman System" {
    [Frontend UI] --> [Talisman Engine API]
    [Talisman Engine API] --> [Natal Chart Parser]
    [Talisman Engine API] --> [Ephemeris Engine]
    [Talisman Engine API] --> [Spirit DB]
    [Talisman Engine API] --> [Transit Analyzer]
    [Talisman Engine API] --> [Election Finder]
    [Talisman Engine API] --> [Design Blueprint Generator]
}

database "Local Data Store" {
    [Natal Charts]
    [Spirit Correspondences]
    [Saved Talismans]
}

[Talisman Engine API] --> [Local Data Store]
@enduml
```

Core Data Models

1. Natal Chart

```
{
  "birth_datetime": "1988-06-21T14:35:00Z",
  "location": "New York, USA",
  "planetary_positions": {
    "Sun": "29° Gemini",
    "Moon": "3° Virgo",
    "Mars": "5° Taurus",
    "...": "..."
  },
  "houses": {...},
  "aspects": {...}
}
```

2. Spirit Entry

```
{
  "name": "Ogun",
  "correspondences": {
    "planet": "Mars",
    "element": "Fire",
    "zodiac": ["Aries", "Scorpio"],
    "virtues": ["Protection", "Courage", "War"],
    "colors": ["Red", "Black"],
    "numbers": [3, 9],
    "shapes": ["Triangle"],
    "days": ["Tuesday"],
    "materials": ["Iron", "Steel"]
  },
  "symbols": {
    "sigil": "ogun.svg",
    "planetary_glyph": "mars.png"
  },
  "description": "Warrior deity of strength and iron"
}
```

Key Algorithms

A.

Transit Analyzer

Checks upcoming planetary transits (next N days) and compares to natal positions. Flags:

- Major transits (conjunction, square, opposition, trine, sextile).
- Benefic (Venus/Jupiter) or Malefic (Mars/Saturn) nature.
- Suggests protective or enhancing talismans accordingly.

Pseudocode:

```
for date in date_range:
    for planet in ephemeris[date]:
        for natal_planet in user_natal_chart:
            if angle_between(planet, natal_planet) in [0, 90, 180, 120,
60]:
                impact = classify_transit(planet, natal_planet)
                suggest_talisman(impact)
```

B.

Election Finder

Evaluates future dates against goal & spirit criteria.

- Favorable signs, dignities, planetary hours, aspects.
- Score-based ranking of potential election moments.

C.

Talisman Blueprint Generator

Combines spirit correspondences + astrology into a symbolic prescription.

Example:

```
{
  "shapes": [{"type": "triangle", "sides": 3, "color": "red"}],
  "symbols": ["mars.png", "ogun.svg"],
  "materials": ["iron"],
  "recommended_date": "2025-08-03T09:23:00",
  "astrological_justification": "Mars rising in Scorpio during Mars hour"
}
```

Implementation

Tech Stack

Layer	Technology	Notes
Frontend	React.js	UI, drag & drop design tool
Backend	Python (FastAPI)	High performance, async-ready API
Astrology	Swiss Ephemeris (via PySwissEph)	Accurate planetary positions
Database	SQLite (MVP) or PostgreSQL	Local data, easy dev setup
Storage	Local filesystem (images, symbols)	Symbol/sigil storage
Charting	AstroPy or flatlib	Natal & transit calculations
Visualization	React Konva or Fabric.js	Canvas-based talisman builder
Packaging	Docker	Containerize app for portability

Directory & File Structure (MVP)

```
astrotalisman/  
├── backend/  
│   ├── main.py                # FastAPI entry point  
│   ├── talisman_logic/  
│   │   ├── ephemeris.py      # Swiss Ephemeris integration  
│   │   ├── natal_chart.py    # Chart calculations  
│   │   ├── transit_analyzer.py  
│   │   ├── election_engine.py  
│   │   └── blueprint_generator.py  
│   ├── models/  
│   │   ├── spirit.py  
│   │   ├── talisman.py  
│   │   └── user_data.py  
│   └── db/  
│       └── database.db  
├── frontend/  
│   ├── components/  
│   │   ├── ChartInputForm.jsx  
│   │   ├── GoalSelector.jsx  
│   │   ├── TalismanDesigner.jsx  
│   │   └── ElectionViewer.jsx  
│   └── App.jsx  
├── assets/  
│   ├── symbols/  
│   │   └── mars.png, ogun.svg  
│   └── colors.json  
└── Dockerfile
```




Key Libraries & APIs

- Swiss Ephemeris – pyswisseph or flatlib for ephemeris data.
 - Flatlib – easy natal/transit parsing.
 - React Konva – shape-based drawing canvas for talisman builder.
 - FastAPI – for fast, interactive data endpoints (e.g., /recommend, /blueprint).
 - Pydantic – for strict schema enforcement of talisman blueprints.
 - SQLite – store natal data, spirits, and saved talismans locally.
-



Talisman Design Tool (MVP Features)

- Choose shape from spirit's correspondences (e.g., triangle).
- Auto-assign number of sides, colors, and overlays.
- Drag/drop glyphs (SVG/PNG) into central layout.
- Export:
 - JSON with config
 - High-res PNG for reference
 - SVG for Illustrator or Inkscape